

CPCS 214-Lab 01/02 Notes

I/O in MIPS on the MARS Simulator

1. Structure of a MIPS program

MIPS code consists of two sections - **.data** and **.text**. The **.data** section is used to describe strings and other data structures. The **.text** section is used to store instructions to be executed.

```
1 | .data
2 |     ...
3 | .text
4 |     ...
```

I/O in MIPS

I/O operations in MIPS are based on the contents of the \$v0 register and executed using the *syscall* command. Values obtained (read) from the user are placed in the register based on the datatype of the input. These values are given in the Table below.

Service	System Call Code (\$v0)	Arguments (in \$a0 or \$f12)
print integer	1	\$a0 (int value to be printed)
print float	2	\$f12 (float)
print double	3	\$f12 (double value to be printed)
print string	4	\$a0 (address of string to be printed)
read integer	5	\$v0 (int read)
read float	6	\$f0 (float read)
read double	7	\$f0 (double read)
read string	8	address of string pointed to by \$a0 and length given in \$a1

2. Printing to display

Printing an *int* to the display

To print an *int* to the display, we need to

1. initialize \$v0 to 1
2. make a call to *syscall*

```
1 | li $v0, 1
2 | syscall
```

When this code is assembled and executed, MARS displays the contents of register \$a0 to the screen.

Printing a *float* to the display

To print a *float* to the display, we need to

1. initialize \$v0 to 2
2. make a call to *syscall*

```
1 | li $v0, 2
2 | syscall
```

When this code is assembled and executed, MARS displays the contents of register \$f12 to the screen.

Printing a *double* to the display

To print a *double* to the display, we need to

1. initialize \$v0 to 3
2. make a call to *syscall*

```
1 | li $v0, 3
2 | syscall
```

When this code is assembled and executed, MARS displays the contents of register \$f12 to the screen.

Displaying a *string*

To display a *string*, we need to do the following

1. Add a line in the **.data** section of the code to initialize a variable name (address) with type *ascii* or *asciiz* and the string within double quotes.
2. Load the address to \$a0 using the command *la*
3. initialize \$v0 to 4
4. make a call to *syscall*

```
1 | .data
2 | message: .ascii "Hello World!"
3 |
4 | .text
5 |
6 | la $a0, message
7 | li $v0, 4
8 | syscall
```

When this code is assembled and executed, MARS displays the content of the address pointed to by the register \$a0 which in this case is *message* which contains the string "Hello World!".

3. Reading input from user

Reading *int* from user

To read an *int* from the user, we need to

1. initialize \$v0 to 5
2. make a call to *syscall*

```
1 | li $v0, 5
2 | syscall
```

When this code is assembled and executed, MARS prompts the user to enter an integer. When the user enters an integer, the program reads it from the keyboard and exits. Before exiting the program, the value read from the user is stored in the register \$v0. This can be seen in the “*Register*” pane on the right of the MARS workspace.

Reading *float* from user

To read a *float* from the user, we need to

1. initialize \$v0 to 6
2. make a call to *syscall*

```
1 | li $v0, 6
2 | syscall
```

When this code is assembled and executed, MARS prompts the user to enter a *float* number. When the user enters a *float*, the program reads it from the keyboard and exits. Before exiting, the value read from the user is stored in the register \$f0. This can be seen in the “*Register*” pane on the right of the MARS workspace.

Reading a *double* from user

To read a *double* from the user, we need to

1. initialize \$v0 to 7
2. make a call to *syscall*

```
1 | li $v0, 7
2 | syscall
```

When this code is assembled and executed, MARS prompts the user to enter a *double* number. When the user enters a *double*, the program reads it from the keyboard and exits. Before exiting, the value read from the user is stored in the register \$f0. This can be seen in the “*Register*” pane on the right of the MARS workspace.

Reading a *string* from user

To read a *string* from the user, we need to do the following

1. Add a line in the **.data** section of the code to initialize a variable name with type *space* and a size to store the string. This needs to be done in the **.data** section of the program. The **.text** section stores the actual code.
2. initialize \$v0 to 8
3. make a call to *syscall*

```
1 | .data
2 | name: .space 64
3 |
4 | .text
5 | la $a0, name
6 | li $a1, 10
7 | li $v0, 8
8 | syscall
```

In the **.data** section, the label *name* is assigned a *space* of 64 bytes. Before reading in a string to *name* we need to store the maximum number of characters to read from the input which in this case we specify as 10.

When this code is assembled and executed, MARS prompts the user to enter a *string*. When the user enters a *string*, the program reads it from the keyboard and exits. Before exiting, the value read from the user is stored in the address pointed to by \$a0, which is *name* in this example.

4. Reading input and displaying it back

To read an *int* from the user, and then displaying it back we need to

1. initialize \$v0 to 5 // to read int
2. make a call to *syscall*
3. move contents from \$v0 to \$a0
4. initialize \$v0 to 1 // to print int
5. make a call to *syscall*

```
1 | # read int from user
2 | li $v0, 5
3 | syscall
4 | move $a0, $v0 # $a0 = $v0
5 |
6 | # display int to user
7 | li $v0, 1
8 | syscall
```

When this code is assembled and executed, MARS prompts the user to enter an integer. When the user enters an integer, the program reads it from the keyboard and stores it in register \$v0 by default. We need to move it to register \$a0 from where it can be printed. When a syscall is made the second time, the program then displays the integer back to the user from register \$a0. This initializing/re-initializing of registers can be observed in the “*Register*” pane on the right of the MARS workspace.